Task 1.4 & 1.5: Nelder–Mead and DIRECT

Objective

Implement and test two derivative-free global/local optimization methods in Python:

(a) **Nelder–Mead** (simplex-based local search with reflection/expansion/contraction/shrink).

(b) **DIRECT** (DIviding RECTangles) global search on a bounded box.

The goal is to learn simplex-based local search and a deterministic global partitioning method, compare their behaviour on multimodal and smooth test problems, and measure computational cost (function evaluations and runtime).

Algorithms to implement

Translate the provided Julia-code into Python. Use clear, well-documented code and ensure deterministic behaviour.

Algorithms (Julia code)

```julia
function nelder_mead(f, S, ϵ; α=1.0, β=2.0, γ=0.5)
    Δ, y_arr = Inf, f.(S)
    while Δ > ϵ
        p = sortperm(y_arr) # sort lowest to highest
        S, y_arr = S[p], y_arr[p]
        xl, yl = S[1], y_arr[1]   # lowest
        xh, yh = S[end], y_arr[end]   # highest
        xs, ys = S[end-1], y_arr[end-1]   # second-highest
        xm = mean(S[1:end-1]) # centroid
        xr = xm + α*(xm - xh) # reflection point
        yr = f(xr)
        if yr < yl
            xe = xm + β*(xr - xm) # expansion point
            ye = f(xe)
            S[end], y_arr[end] = ye < yr ? (xe, ye) : (xr, yr)
        elseif yr ≥ ys
            if yr < yh
                xh, yh, S[end], y_arr[end] = xr, yr, xr, yr
            end
            xc = xm + γ*(xh - xm) # contraction point
            yc = f(xc)
            if yc > yh
                for i in 2 : length(y_arr)
                    S[i] = (S[i] + xl)/2 # shrinkage
                    y_arr[i] = f(S[i])
                end
            else
                S[end], y_arr[end] = xc, yc
            end
        else
            S[end], y_arr[end] = xr, yr
        end
        Δ = std(y_arr, corrected=false)
    end
    return S[argmin(y_arr)]
end
```

```julia
function direct(f, a, b, ϵ, k_max)
    # Input: objective function f, vector of lower bounds a,
    # vector of upper bounds b, tolerance parameter ϵ, number of iterations k_max.
    g = reparameterize_to_unit_hypercube(f, a, b) # normalize domain
```

```
    intervals = Intervals() # interval storage
    n = length(a) # problem dimension
    c = fill(0.5, n) # center of domain
    interval = Interval(c, g(c), fill(0, n)) # initial interval
    add_interval!(intervals, interval) # add to structure
    c_best, y_best = copy(interval.c), interval.y # best known point so far
    for k in 1:k_max # main loop
        S = get_opt_intervals(intervals, ϵ, y_best) # select optimal intervals
        to_add = Intervals.empty_list() # prepare new intervals
        for interval in S
            append!(to_add, divide(g, interval)) # add new intervals
            dequeue!(intervals[vertex_dist(interval)]) # remove old interval
        end
        for interval in to_add # iterate through new intervals
            add_interval!(intervals, interval) # insert into data structure
            if interval.y < y_best # update best solution if needed
                c_best, y_best = copy(interval.c), interval.y
            end
        end
    end
    return rev_unit_hypercube_parameterization(c_best, a, b)
end

function reparameterize_to_unit_hypercube(f, a, b)
    Δ = b - a
    return x -> f(x.* Δ + a) # scale and shift to map from [0,1]^n to [a,b]
end

function rev_unit_hypercube_parameterization(x, a, b)
    return x.* (b - a) + a # map from unit hypercube back to original space
end
```

```
using DataStructures # import priority queues and dictionaries

struct Interval
    c     # center
    y     # function value at center
    depths # division depth for each coordinate
end

min_depth(interval) = minimum(interval.depths) # smallest depth value

# Euclidean distance from center to vertex
vertex_dist(interval) = norm(0.5 * 3.0.^ (-interval.depths), 2)

# main data structure: maps distances to priority queues of intervals
const Intervals = OrderedDict{Float64, PriorityQueue{Interval, Float64}}()

function add_interval!(intervals, interval)
    d = vertex_dist(interval)
    if !haskey(intervals, d)
        intervals[d] = PriorityQueue{Interval, Float64}()
    end
    return enqueue!(intervals[d], interval, interval.y)
end

function get_opt_intervals(intervals, ϵ, y_best)
    stack = Interval[] # Stack for storing potentially optimal intervals
    for (x, pq) in intervals # Iterate over interval groups by distance
        if !isempty(pq)
            interval = DataStructures.peek(pq)[1] # Interval with best value
            y = interval.y
            # Check dominance against last two in stack
```

```
            while length(stack) > 1
                interval1 = stack[end]
                interval2 = stack[end-1]
                x1, y1 = vertex_dist(interval1), interval1.y
                x2, y2 = vertex_dist(interval2), interval2.y
                ℓ = (y2 - y) / (x2 - x) # Slope of lower bound line
                if y1 <= ℓ*(x1 - x) + y + ϵ
                    break # Interval is dominated - stop checking
                end
                pop!(stack) # Remove dominated interval
            end
            if !isempty(stack) && interval.y > stack[end].y + ϵ
                continue # New interval is worse - skip it
            end
            push!(stack, interval) # Add new potentially optimal interval
        end
    end
    return stack
end

function divide(f, interval)
    # center, min depth, dimension
    c, d, n = interval.c, min_depth(interval), length(interval.c)
    # returns vector of index for elements with minimal depth
    dirs = findall(interval.depths.== d)
    # new centers along each direction
    cs = [(c + 3.0^(-d-1) * basis(i, n), c - 3.0^(-d-1) * basis(i, n)) for i in dirs]
    # returns list of pairs of function values at new centers
    vs = [(f(C[1]), f(C[2])) for C in cs]
    # returns list of smallest values from each pair in vs
    minvals = [min(V[1], V[2]) for V in vs]
    intervals = Interval[]
    # copy(...) → returns a shallow copy of the depths vector
    depths = copy(interval.depths)
    # sortperm(...) → returns indices that would sort minvals
    for j in sortperm(minvals)
        depths[dirs[j]] += 1
        C, V = cs[j], vs[j]
        # push! → appends new Interval objects to the list
        push!(intervals, Interval(C[1], V[1], copy(depths)))
        push!(intervals, Interval(C[2], V[2], copy(depths)))
    end
    # original interval is also added with updated depths
    push!(intervals, Interval(c, interval.y, copy(depths)))
    return intervals
end
```

Nelder–Mead (summary). Implement the classical Nelder–Mead method with parameters α (reflection), β (expansion), and γ (contraction). Create the initial simplex S from the given starting point $x^{(0)} \in \mathbb{R}^n$. In particular, the algorithm requires $n+1$ distinct vertices (a non-degenerate simplex), so construct S by taking $x^{(0)}$ itself and adding small coordinate perturbations. For example, in 2D with a small scalar $\delta > 0$:

$$S = \{\, x^{(0)},\ x^{(0)} + \delta e_1,\ x^{(0)} + \delta e_2 \,\},$$

where e_i are the standard basis vectors. In general, form $x^{(0)}$ plus one perturbed copy per coordinate to obtain $n+1$ vertices. This ensures a valid geometric simplex for reflection, expansion, contraction and shrink operations.

Stop when the standard deviation of the function values at the simplex vertices falls below ϵ, or after a reasonable maximum number of iterations. You may follow the lecture code (reflection → expansion / contraction / shrink) exactly.

DIRECT (summary). Implement a DIRECT-like algorithm on a box $[a, b]^n$:

- Reparameterize the box to the unit hypercube $[0,1]^n$.
- Maintain a set of hyper-rectangles (intervals) with stored centers, depths and function values.
- At each iteration select potentially optimal intervals (by the selection rule described in the lecture/pseudocode), subdivide them along minimal-depth coordinates, evaluate new centers, update the set and the best-known point.
- Stop after $k_{\max}$ subdivisions or when a tolerance criterion with parameter ε is met.

You may simplify data structures (e.g. use lists + sorting) but keep the core selection/division logic intact.

Test functions

Apply both algorithms to the following benchmark functions (2D):

- **Ackley**:

$$f(x,y) = -20\exp\Big(-0.2\sqrt{\tfrac{1}{2}(x^2+y^2)}\Big) - \exp\Big(\tfrac{1}{2}(\cos(2\pi x)+\cos(2\pi y))\Big) + e + 20,$$

  with global minimum at $(0,0)$ and $f(0,0)=0$.

- **Branin**:

$$f(x,y) = \Big(y - \tfrac{5.1}{4\pi^2}x^2 + \tfrac{5}{\pi}x - 6\Big)^2 + 10\Big(1-\tfrac{1}{8\pi}\Big)\cos(x) + 10,$$

  with multiple global minima (one example (π, 2.275) with $f \approx 0.397887$.).

- **Rosenbrock**:

$$f(x,y) = (1-x)^2 + 5(y-x^2)^2,$$

  with global minimum at $(1,1)$ and $f(1,1)=0$.

- **Rastrigin**:

$$f(x,y) = 20 + (x^2 - 10\cos(2\pi x)) + (y^2 - 10\cos(2\pi y)),$$

  with global minimum at $(0,0)$ and $f(0,0)=0$.

(You may add other functions from previous tasks for additional experiments.)

Domains and starting data

- For **Nelder–Mead** use the following starting points and construct the initial simplex from each start (e.g. use coordinate perturbation $\delta = 0.05$):
  - Ackley: $x^{(0)} = (0,1)$.
  - Branin: $x^{(0)} = (2.0, 2.0)$.
  - Rosenbrock: $x^{(0)} = (-1.5, 2.0)$.
  - Rastrigin: $x^{(0)} = (2.5, 2.5)$.

  Tolerance parameter $\epsilon = 10^{-6}$ and max iteration $k_{\max} = 100$.

- For **DIRECT** use the following bounds and parameters:
  - Domain (apply to all test functions):
    - Ackley: $a = [-2,-3]$, $b = [4,3]$.
    - Branin: $a = [0,0]$, $b = [4,4]$.
    - Rosenbrock: $a = [-5,0]$, $b = [2,4]$.
    - Rastrigin: $a = [-1,-1]$, $b = [6,6]$.
  - Tolerance parameter ε and max subdivisions $k_{\max} = 150$.

Experiment protocol and outputs

For each combination (algorithm, function, start / domain) perform a single deterministic run. Keep stopping criteria consistent: tolerance and maximum number of outer iterations.

Save/produce the following artifacts:

1. **Results (console):** a table with columns

   `function, start, method, x_found, f(x_found), iterations, f_evals, time_s`

   where `iterations` denotes algorithm outer iterations (Nelder–Mead simplex updates / DIRECT main-iterations), `f_evals` is total function evaluations, and `time_s` is runtime in seconds.

2. **Trajectory plots:**
   - for Nelder–Mead produce contour plots with simplex vertices and the path of the simplex centroid (save as `plot_<func>_nelder_trajectory.png`).
   - For DIRECT produce a plot showing sampled centers (scatter) and location of best found point (save as `plot_<func>_direct_samples.png`).

3. **Code:** Python source implementing both algorithms and helper functions.

Comparative check

- Compare Nelder–Mead and DIRECT on the same test functions: discuss final objective, number of function evaluations, and runtime.
- Discuss where DIRECT finds better global minima (multimodal surfaces like Ackley, Rastrigin) and where Nelder–Mead is efficient (smooth unimodal / local refinement).

Tasks (student checklist)

1. Implement Nelder–Mead and DIRECT in Python following the provided pseudocode.
2. Use consistent stopping tolerances and max iterations.
3. Run the experiments on the listed test functions and starting points/domains.
4. Produce the requested plots and console table.
5. Write a short comparison (3–5 sentences) answering:
   - Which method found a lower objective on each function and why.
   - Which method used fewer function evaluations and why.
   - Where each method is preferable (local refinement vs global search).

Deliverables

- Python code implementing Nelder–Mead and DIRECT.
- Trajectory plots as described.
- Console table with results for all runs (include runtime and evaluation counts).
- Short written discussion (3–5 sentences).

Additional (advanced) exercises — choose one

1. **Adaptive DIRECT:** implement an improved interval-selection rule (use alternative dominance rule) and compare performance.

   Explanation: Biased selection: *Give higher priority to intervals with both small function values and relatively large size (e.g. minimize $f(c) - \lambda \cdot d$ where d is interval diameter, $\lambda > 0$ a tunable parameter).*

   Greedy selection: *Always include not only the standard potentially optimal intervals, but also the interval with the current best function value, regardless of dominance tests.*

   Run the modified algorithm and compare convergence against the classical DIRECT.

2. **Hybrid local-global:** run DIRECT for k iterations to identify promising regions, then run Nelder–Mead initialized at best DIRECT centers; compare the hybrid to plain methods.

3. **High-dimension experiment:** test DIRECT and Nelder–Mead on 4D or 6D variants of Ackley/Rastrigin and report scaling of function evaluations and runtime.

   Explanation: Use the following standard n-dimensional definitions:

   - **Ackley function** (n-D):

   $$f(x) = -20 \exp\left(-0.2 \sqrt{\frac{1}{n} \sum_{i=1}^{n} x_i^2} \right) - \exp\left(\frac{1}{n} \sum_{i=1}^{n} \cos(2\pi x_i) \right) + 20 + e,$$

   with global minimum at $x^* = (0, \ldots, 0)$ and $f(x^*) = 0$.

   - **Rastrigin function** (n-D):

   $$f(x) = 10n + \sum_{i=1}^{n} \left(x_i^2 - 10 \cos(2\pi x_i) \right),$$

   with global minimum at $x^* = (0, \ldots, 0)$ and $f(x^*) = 0$.

Notes and practical hints

- Implement careful bookkeeping of function evaluations (count every call).
- For Nelder–Mead, document how you construct the initial simplex (report δ used).
- For DIRECT, you may use a simplified data structure (sorted list) but preserve the selection/division logic.
- Use sensible plotting domains to show relevant behaviour (for Direct show sampled centers as scatter; for NM show simplex vertices).
- Ensure deterministic runs (no randomized seeds affecting results).